

中山大學
SUN YAT-SEN UNIVERSITY

Lecture 11
Sorting

Data Structures and Algorithms

Zizhen Zhang, School of Computer Science and Engineering, Sun Yat-sen University

zhangzizhen@gmail.com

Outline

- *Introduction and Notation*
- *Insertion Sort*
- *Selection Sort*
- *Shell Sort*
- *Bucket Sort*
- *Radix Sort*
- ----------
- Divide-and-Conquer Sorting
- Mergesort
- Quicksort

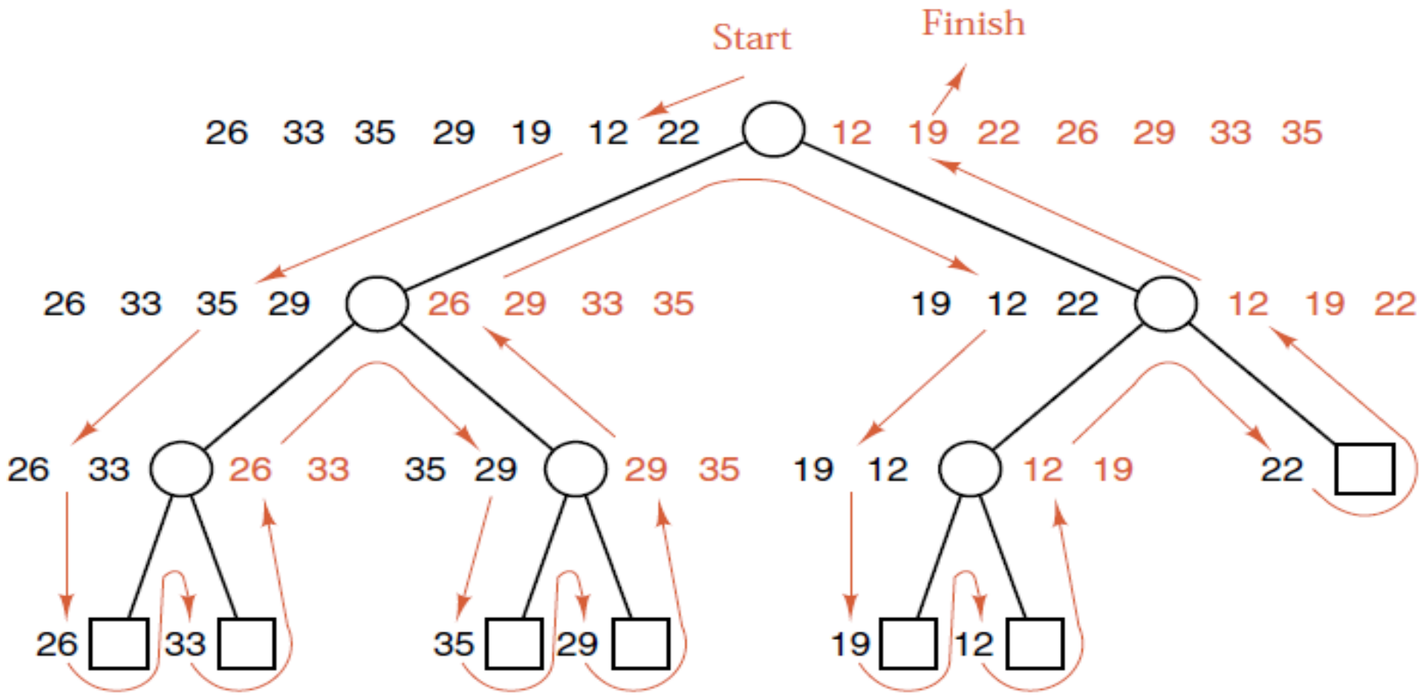
Divide and Conquer Sorting

- The main ideas:
  - It's easier to sort short lists;
  - So we can divide the list into sublists and sort the sublists;
  - Then we combine the sorted sublists into a sorted list;
- How to sort the sublists? One way is to use the same method. This process will terminate because a list of one entry is sorted.
- Question: How to divide a list into two sublists and then combine sorted lists?

```
void Sortable_list :: sort( )
{
   if the list has length greater than 1 {
      partition the list into lowlist, highlist;
      lowlist.sort( );
      highlist.sort( );
      combine(lowlist, highlist);
   }
}
```

Mergesort

- It was invented by *John von Neumann* in 1945.
- Implementation
  - If the size of the list is greater than 1, chop(拆分) the list into two sublists of equal sizes.
  - Sort two sublists separately.
  - Merge the two sorted sublists into a single sorted list.

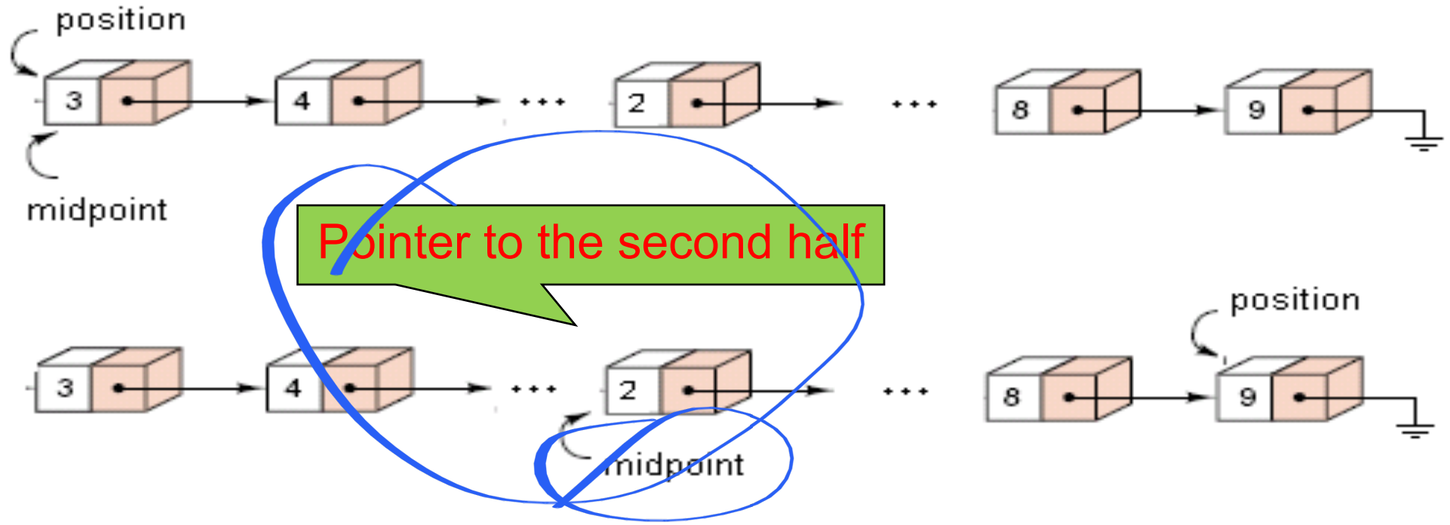


Mergesort for Linked Lists

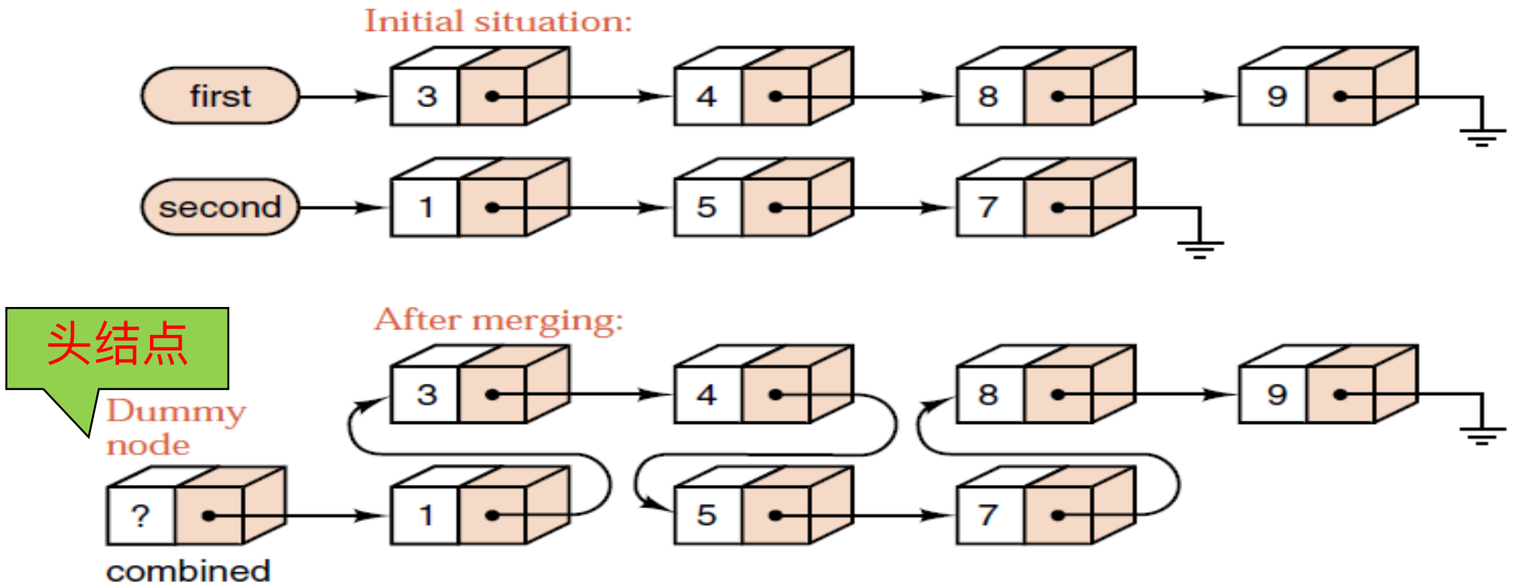
```
template <class Record>
void Sortable_list<Record> ::
      recursive_merge_sort(Node<Record> * &sub_list)
/* Post: The nodes referenced by sub_list have been rearranged so that their keys
         are sorted into nondecreasing order. The pointer parameter sub_list is
         reset to point at the node containing the smallest key.
   Uses: The linked List implementation of Chapter 6; the functions divide_from,
         merge, and recursive_merge_sort. */
{
  if (sub_list != NULL && sub_list->next != NULL) {
     Node<Record> *second_half = divide_from(sub_list);
     recursive_merge_sort(sub_list);
     recursive_merge_sort(second_half);
     sub_list = merge(sub_list, second_half);
  }
}
```

Mergesort for Linked Lists

Divide:



Merge:

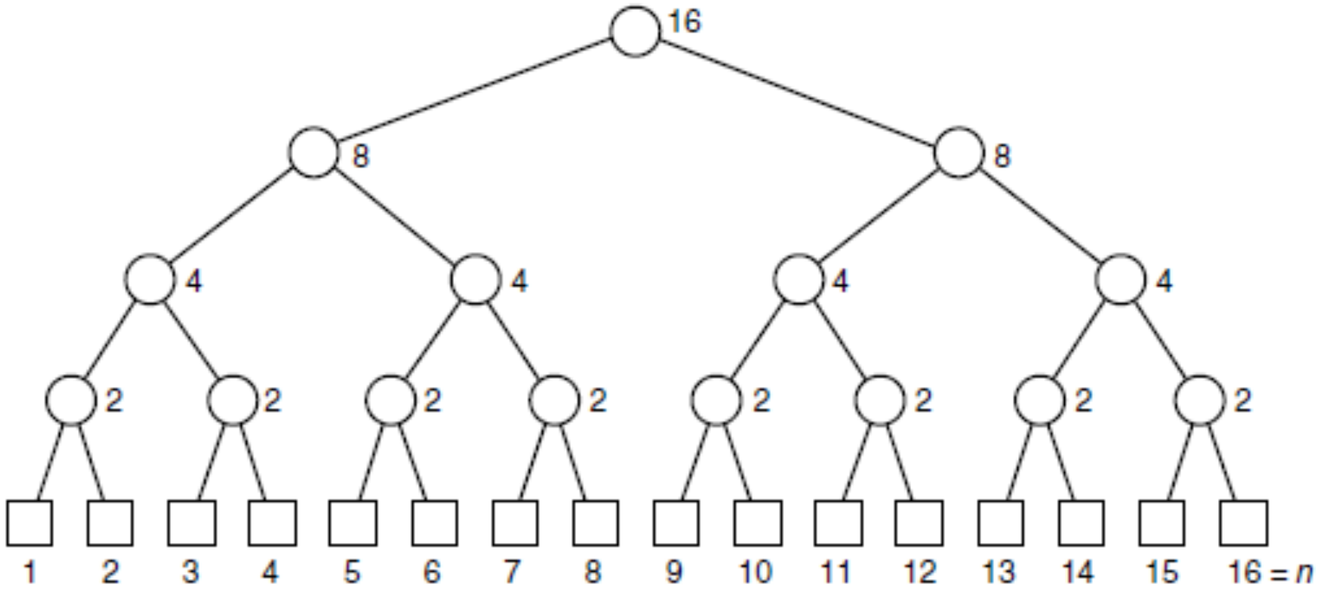


Merge Two Sorted Linked Lists

```
template <class Record>
Node<Record> *Sortable_list<Record> :: merge(Node<Record> *first,
                                             Node<Record> *second)
/* Pre:   first and second point to ordered lists of nodes.
   Post:  A pointer to an ordered list of nodes is returned. The ordered list contains
          all entries that were referenced by first and second. The original lists of
          nodes referenced by first and second are no longer available.
   Uses:  Methods for Record class; the linked List implementation of Chapter 6
{
   Node<Record> *last_sorted;   //   points to the last node of sorted list
   Node<Record> combined;       //   dummy first node, points to merged list
   last_sorted = &combined;
   while (first != NULL && second != NULL) {  //   Attach node with smaller key
      if (first->entry <= second->entry) {
         last_sorted->next = first;
         last_sorted = first;
         first = first->next;        //   Advance to the next unmerged node.
      }
      else {
         last_sorted->next = second;
         last_sorted = second;
         second = second->next;
      }
   }
//   After one list ends, attach the remainder of the other.
   if (first == NULL)
      last_sorted->next = second;
   else
      last_sorted->next = first;
   return combined.next;
}
```

Analysis of Mergesort

- On each level, the number of entries to be merged is *n*;
- The number of comparisons on each level except the level on leaves is at most *n*;
- The height of the recursion tree is $\lceil \lg n \rceil$, so the total number of comparisons is less than $\lceil n \lg n \rceil$.



Analysis of Mergesort

- Number of comparisons(average): *nlgn-1.1583n+1*
- Time complexity: *O(n logn)* for both average-case and worst-case
- Space complexity: *O(n),* or *O(1)* for in-place mergesort
- Stability: stable

Mergesort Application: Inverse number

- 逆序对：对于一个包含N个非负整数的数组A[1..n]，如果有i < j，且A[i]>A[j]，则称(i, j)为数组A中的一个逆序对。
- 例如，数组(3, 1, 4, 5, 2)的逆序对有(3,1), (3,2), (4,2), (5,2)，共4个。
- 枚举：$O(n^2)$. n <= 5000适用
- 归并排序：
  - Divide: 把序列等分成两半
  - Conquer: 统计*i*和*j*都在左边或者均在右边的逆序对个数。
  - Combine: 统计*i*在左边，但j在右边的逆序对个数。
  - Q: 如何求出*i*在左边，而*j*在右边的逆序对个数呢?
  - A: 对于右边的每个*j*，统计左边比它大的元素个数*f(j)*，则所有*f(j)*之和就是答案。
  - Time complexity: *O(n logn)*

Inverse number

```
int inverse_num = 0;
void inverse_number(int *A, int x, int y, int* T)
{
   if (y-x > 1)
   {
       int m = x + (y-x) / 2;  // divide
       int p = x, q = m, i = x;
       inverse_number(A, x, m, T);    // 递归求解
       inverse_number(A, m, y, T);    // 递归求解
       while (p < m || q < y)
       {
           if(q >= y || (p < m && A[p] <= A[q]))
               T[i++] = A[p++];
           else {T[i++] = A[q++]; reverse_num += m-p;}
       }
       for(i=x; i<y; i++) A[i] = T[i];
   }
}
```

Quicksort

- We first choose some key from the list for which, we hope, about half the keys will come before and half after. Call this key the ***pivot***.
- Then we partition the items so that all those with keys less than the pivot come in one sublist, and all those with greater keys come in another.
- Then we sort the two reduced lists separately, put the sublists together, and the whole list will be in order.

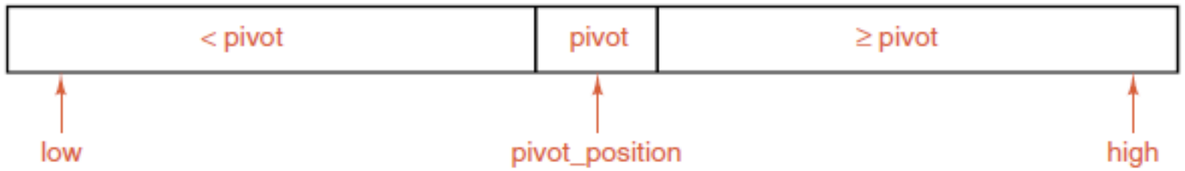
Quicksort for Contiguous Lists

- **Recursive function:**

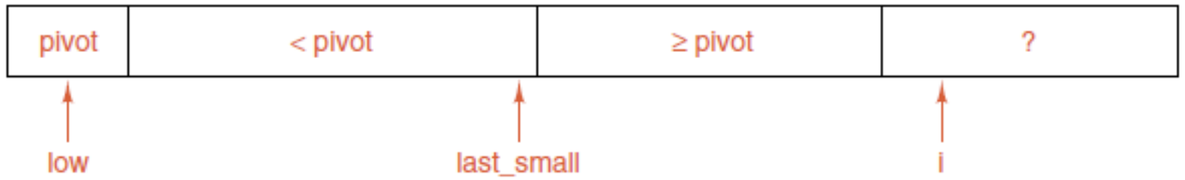
```
template <class Record>
void Sortable_list<Record> :: recursive_quick_sort(int low, int high)
/* Pre:   low and high are valid positions in the Sortable_list.
   Post:  The entries of the Sortable_list have been rearranged so that their keys
          are sorted into nondecreasing order.
   Uses:  Contiguous List implementation of Chapter 6, recursive_quick_sort,
          and partition.  */
{
   int pivot_position;
   if (low < high) {                    //   Otherwise, no sorting is needed.
      pivot_position = partition(low, high);
      recursive_quick_sort(low, pivot_position - 1);
      recursive_quick_sort(pivot_position + 1, high);
   }
}
```

Partitioning the list

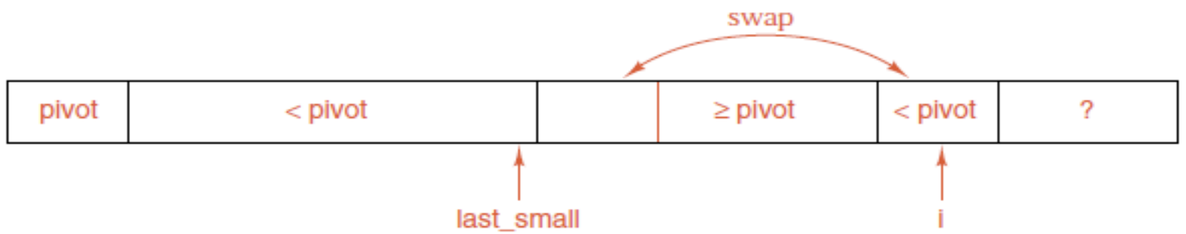
- **Goal:**



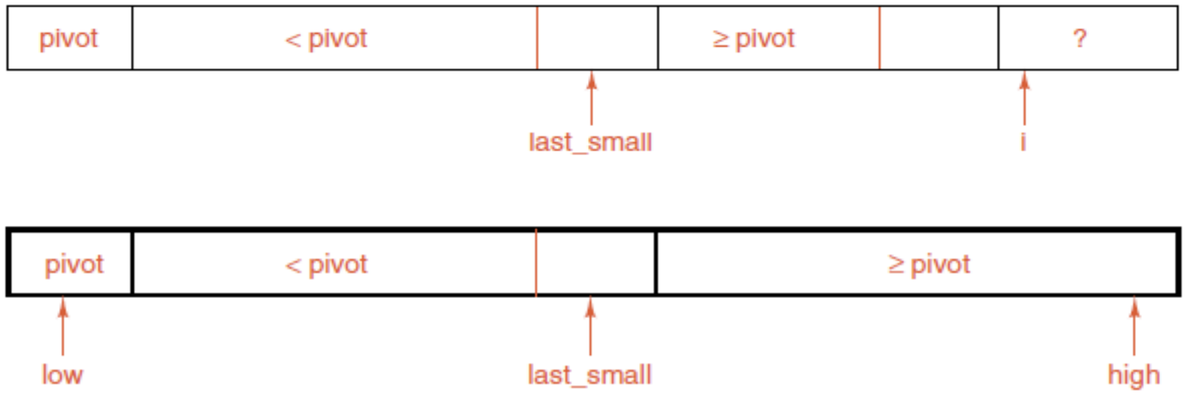
- **Loop invariant:**



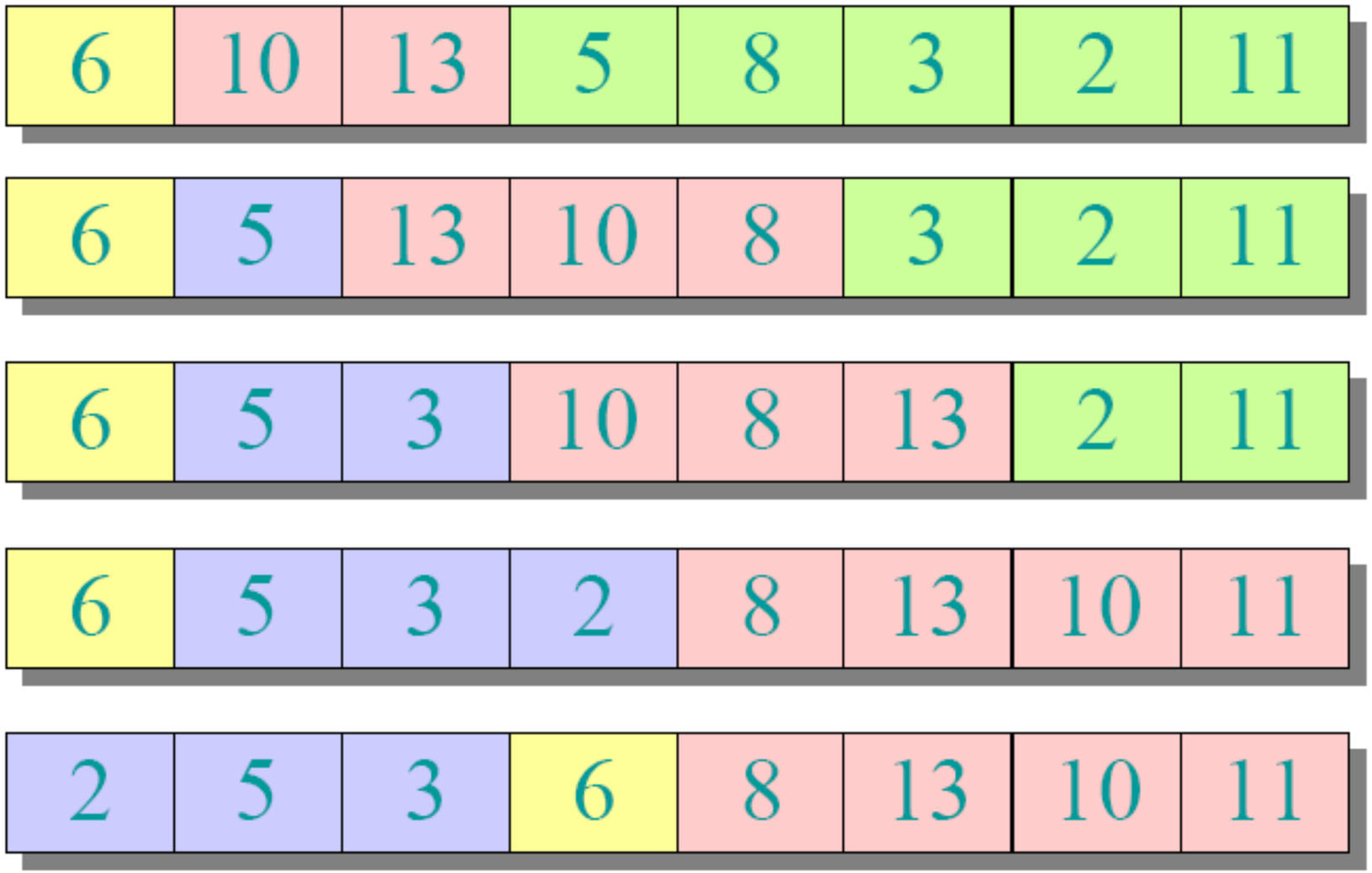
- **Restore the invariant:**



- **Final position:**



Partitioning Example



Lomuto's Partitioning

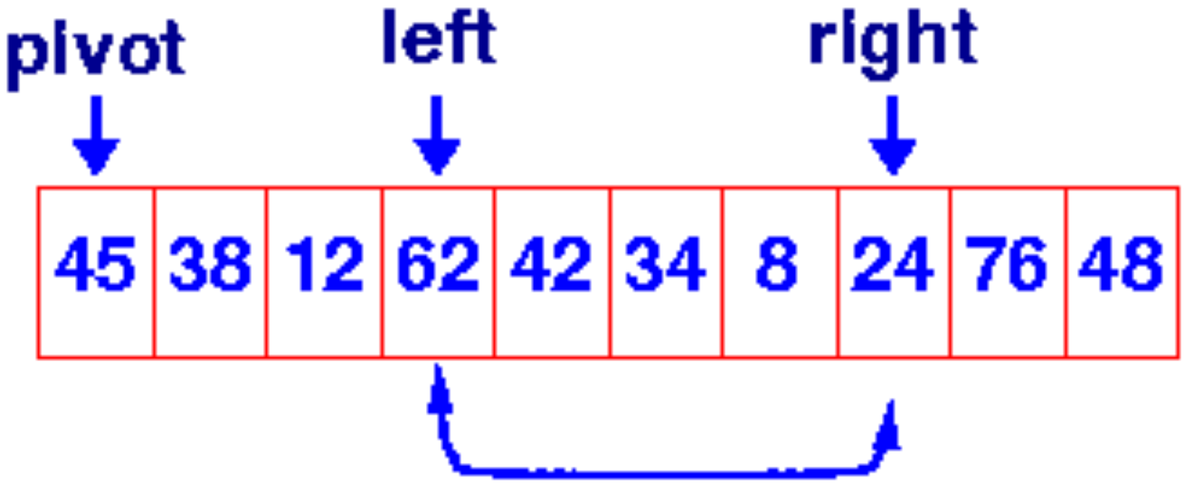
```
template <class Record>
int Sortable_list<Record> :: partition(int low, int high)
{
   Record pivot;
   int i,                          //   used to scan through the list
       last_small;                 //   position of the last key less than pivot
   swap(low, (low + high)/2);
   pivot = entry[low];             //   First entry is now pivot.
   last_small = low;
   for (i = low + 1; i <= high; i++)

/* At the beginning of each iteration of this loop, we have the following conditions:
                If low < j <= last_small then entry[j].key < pivot.
                If last_small < j < i then entry[j].key >= pivot. */

      if (entry[i] < pivot) {
         last_small = last_small + 1;
         swap(last_small, i);      //   Move large entry to right and small to left.
      }
   swap(low, last_small);          //   Put the pivot into its proper position.
   return last_small;
}
```

Another Partitioning Implementation

- 两个指针left和right. left左边的都小于等于x, right右边的都大于x, **未知区域在中间**
- left和right交替移动, 一旦发现不满足要求的元素就停下来. 交换这两个元素, 使同时满足要求
- 好处：当相同元素比较多时较快



Hoare's Partitioning

```
int partition(int a[], int low, int high )
{
    int left, right;
    int pivot_item = a[low];
    left = low;
    right = high;
    while ( left < right ) {
        /* Move left while item <= pivot */
        while( a[left] <= pivot_item && left < high ) left++;
        /* Move right while item > pivot */
        while( a[right] > pivot_item && low < right ) right--;
        if ( left < right ) {
                    int tmp = a[left];
                    a[left] = a[right];
                    a[right] = tmp;
            }
    }
    /* right is final position for the pivot */
    a[low] = a[right];
    a[right] = pivot_item;
    return right;
}
```

Analysis of Quicksort

- **Worst-case analysis:**
  - If the pivot is chosen poorly, one of the partitioned sublists may be empty and the other reduced by only one entry. In this case, quicksort is slower than either insertion sort or selection sort.
- **Choice of pivot:**
  - *First or last entry:* Worst case appears for a list already sorted or in reverse order.
  - *Central entry:* Poor cases appear only for unusual orders.
  - *Random entry:* Poor cases are very unlikely to occur.

Analysis of Quicksort

- The number of total comparisons:

$$C(n) = n-1 + C(r) + C(n-r-1)$$

- Let C(n) be the number of comparisons of keys made by quicksort when applied to a list of length n. We have C(1) = C(0) = 0. The partition function compares the pivot with every other key in the list exactly once, and thus the function partition accounts for exactly n − 1 key comparisons, where we assume the list is split into lists of r entries and n - r - 1 entries.
- The number of comparisons of the worst case: the worst case is when the pivot cannot split the list into two lists, and one list has n-1 entries, that is, when the list is in increasing order.

$$C(n) = n-1 + C(n-1)$$

- Solving this recurrence relation: $C(n) = \frac{1}{2} n^2 - \frac{1}{2} n$

Analysis of Quicksort

- To do the average-case analysis, we shall assume that all possible orderings of the list are equally likely, and for simplicity, we take the keys to be just the integers from 1 to n.
- If the pivot is the p-th smallest key, let C(n,p) be the average number of key comparisons done.

$$C(n,p)=n-1+C(p-1)+C(n-p)$$

- Averaging from p = 1 to p = n gives, for n >= 2,

$$C(n)=n-1+2/n\ (C(0)+C(1)+\ldots+C(n-1))$$

In its average case, quicksort performs

$$C(n) = 2n \ln n + O(n) \approx 1.39n \lg n + O(n)$$

comparisons of keys in sorting a list of n entries in random order.

Application: Selection Problem

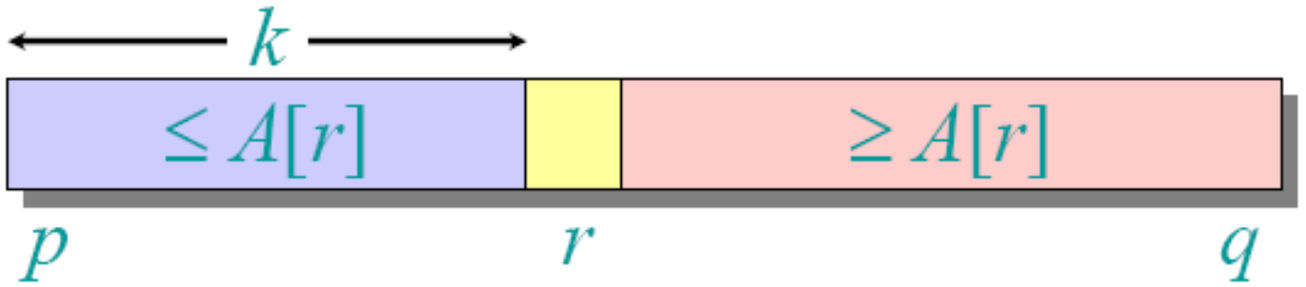
- Problem description:
  - Given array A[1..n], find the **k-th smallest element** in A

- Refer to std::nth_element();

```
template <class RandomAccessIterator>
  void nth_element (RandomAccessIterator first, RandomAccessIterator nth,
                    RandomAccessIterator last);
```

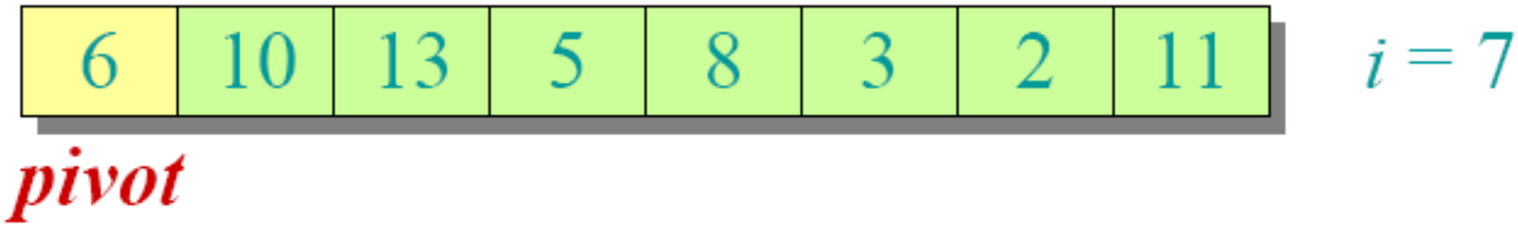
Randomized Selection Algorithm

```
RAND-SELECT(A, p, q, i)        ▷ ith smallest of A[p..q]
    if p = q then return A[p]
    r ← RAND-PARTITION(A, p, q)
    k ← r − p + 1              ▷ k = rank(A[r])
    if i = k then return A[r]
    if i < k
        then return RAND-SELECT(A, p, r − 1, i)
        else return RAND-SELECT(A, r + 1, q, i − k)
```

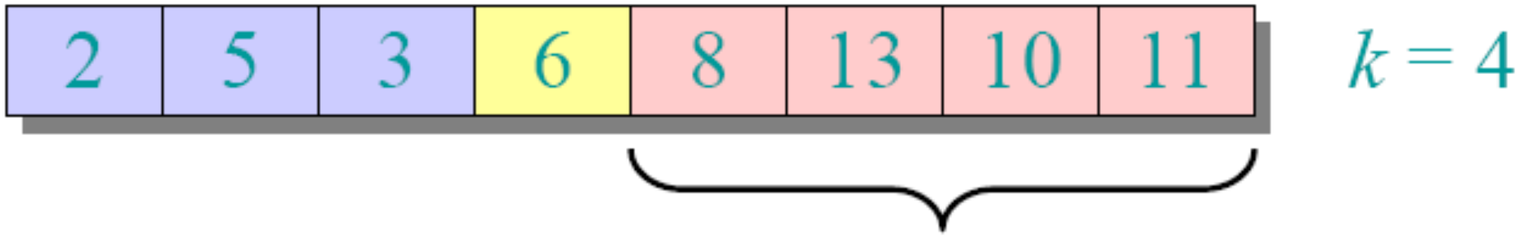


Randomized Selection Algorithm

Select the $i = 7$th smallest:



Partition:



Select the $7 - 4 = 3$rd smallest recursively.

Analysis of Randomized Selection

- **Worst case:** A[r] = smallest or largest element
- Time: T(n) = O(n) (for splitting) + T(n-1)
- Solving the recurrence, $T(n) = O(n^2)$
- Pr(Worst Case) = $\frac{2}{n} \cdot \frac{2}{n-1} \cdot \ldots \frac{2}{2} \approx \frac{2^n}{n!}$
- **Best case:** A[r] is the k-th element
- Time taken T(n) = O(n) (for splitting) + O(1)
- T(n) = O(n)
- Pr(Best Case) >= 1/n

Analysis of Randomized Selection

- **Average case:** Let T(n) be the expected running time on an array of size n
- **Lucky split:** A[r] is the m-th smallest element, for n/4 <= m <= 3n/4. Pr[Lucky Split] = 1/2
- T(n) <= Time to split + Pr[Lucky Split] x T(array of size <= 3n/4) + Pr[Unlucky Split] x T(array of size <= n)

  <= n + (1/2) T(3n/4) + (1/2) T(n)
- Solving, T(n) <= T(3n/4) + 2n = O(n)

- Question: Is there a linear selection algorithm for every case?

Thank you!

